



中国海洋大学
OCEAN UNIVERSITY OF CHINA

系统开发工具基础实验报告

第 1 周：Shell、Git 与 LaTeX 基础实践

姓名： 姬艺华

学号： 25020007046

日期： 2026 年 8 月 24 日

目录

1	课上习题	3
1.1	第 1 题：含空格文件名的批量整理	3
1.1.1	任务说明	3
1.1.2	关键操作	3
1.1.3	实验结果	4
1.2	第 2 题：随机访问日志统计	5
1.2.1	任务说明	5
1.2.2	脚本实现	5
1.2.3	实验结果	6
1.3	第 3 题：制造、解决并解释一次合并冲突	7
1.3.1	任务说明	7
1.3.2	关键过程	7
1.3.3	实验结果	8
1.4	第 4 题：修复并构建一页技术说明	8
1.4.1	任务说明	9
1.4.2	核心实现	9
1.4.3	构建与结果	9
2	课下习题	10
2.1	检查 Shell 环境	10
2.2	ls -l 与文件权限	11
2.3	Glob 匹配	11
2.4	Shell 引号	12
2.5	标准流重定向	12
2.6	与退出状态	13
2.7	cd 为什么是内建命令	14
2.8	判断文件是否存在	14
2.9	文件执行权限	15
2.10	set -x	16
2.11	-- 的作用	16
3	Git 与作业提交	17
3.1	仓库内容整理	17
3.2	GitHub / Gitee 提交记录	17
3.3	commit 记录截图、仓库页面截图	18

4 问题以及经验	18
4.1 实验中遇到的问题	18
4.2 实验经验	19
4.3 总结	19

1 课上习题

本模块记录上课实验四道例题操作基本过程，以及关键点。

1.1 第 1 题：含空格文件名的批量整理

主题：Shell 基础与文件系统

1.1.1 任务说明

创建 q01，使用命令行完成目录与文件创建、隐藏文件查看、按扩展名复制文件、权限设置以及清单生成。

1.1.2 关键操作

使用 `mkdir -p` 创建 `input` 文件夹以及所包含的文件；使用 `printf “...”` 输入文件内容；使用 `touch` 创建文件；使用 `ls -laR` 查看 `input` 中所有文件：

```
mkdir -p input/docs input/tmp
printf "alpha\nbeta\n" > "input/docs/notes one.txt"
printf "hidden\n" > input/docs/.secret.txt
touch input/tmp/empty.txt
printf "start\nfinish\n" > input/run.log
ls -laR input
```

随后查找所有 `.txt` 文件，并在复制时保留相对目录结构：

创建 `work/25020007046` 文件夹；使用 `find` 找寻所有 `.txt` 类型的文件；使用 `-exec` 找到一个文件后执行 `cp` 复制操作；`-parents` 保留原来的路径结构：

```
mkdir -p work/25020007046
find input -type f -name "*.txt" \
    -exec cp --parents "{}" work/25020007046 \;
```

设置目录与普通文件权限，并生成文件清单：

使用 `chmod` 改变文件权限；`-type d` 找目录，`-type f` 找文件；`%P` 会保留相对路径；`%s` 显示文件大小：

```
find work/25020007046 -type d -exec chmod 750 {} \;
find work/25020007046 -type f -exec chmod 640 {} \;
find work/25020007046 -type f \
    -printf "%P %s\n" > inventory.txt
cat inventory.txt
```

关键点：

- 文件名中包含空格时，要用双引号把完整路径括起来；查看隐藏文件要使用 `ls -a`。
- `ls -laR input` 中，`-l` 是长格式，`-a` 显示隐藏文件，`-R` 递归查看子目录。

- `find` 用来查找文件, `cp --parents` 可以在复制时保留原来的目录结构。
- 权限数字由可读 `r=4`、可写 `w=2`、可进入 `x=1` 相加得到; 本目录使用 750, 普通文件使用 640。

1.1.3 实验结果

```
haru@haru-VMware-Virtual-Platform:~/q01$ printf "hidden" > input/docs/.secret.txt
haru@haru-VMware-Virtual-Platform:~/q01$ ls -a input/docs
.  ..  'notes one.txt'  .secret.txt
haru@haru-VMware-Virtual-Platform:~/q01$ touch input/tmp/empty.txt
haru@haru-VMware-Virtual-Platform:~/q01$ ls -l input/tmp
总计 0
-rw-rw-r-- 1 haru haru 0  8月 24 16:06 empty.txt
haru@haru-VMware-Virtual-Platform:~/q01$ printf "start\nfinish\n" > input/run.log
haru@haru-VMware-Virtual-Platform:~/q01$ ls -laR input
input:
总计 20
drwxrwxr-x 4 haru haru 4096  8月 24 16:07 .
drwxrwxr-x 3 haru haru 4096  8月 24 15:32 ..
drwxrwxr-x 2 haru haru 4096  8月 24 16:06 docs
-rw-rw-r-- 1 haru haru  13  8月 24 16:07 run.log
drwxrwxr-x 2 haru haru 4096  8月 24 16:06 tmp

input/docs:
总计 16
drwxrwxr-x 2 haru haru 4096  8月 24 16:06 .
drwxrwxr-x 4 haru haru 4096  8月 24 16:07 ..
-rw-rw-r-- 1 haru haru  11  8月 24 15:57 'notes one.txt'
-rw-rw-r-- 1 haru haru   6  8月 24 16:06 .secret.txt

input/tmp:
总计 8
drwxrwxr-x 2 haru haru 4096  8月 24 16:06 .
drwxrwxr-x 4 haru haru 4096  8月 24 16:07 ..
-rw-rw-r-- 1 haru haru   0  8月 24 16:06 empty.txt
haru@haru-VMware-Virtual-Platform:~/q01$
```

图 1 q01 输入目录与隐藏文件检查结果

```
haru@haru-VMware-Virtual-Platform:~/q01$ ls -laR work/25020007046
work/25020007046:
总计 12
drwxr-x--- 3 haru haru 4096  8月 24 16:17 .
drwxrwxr-x 3 haru haru 4096  8月 24 16:16 ..
drwxr-x--- 4 haru haru 4096  8月 24 16:17 input

work/25020007046/input:
总计 16
drwxr-x--- 4 haru haru 4096  8月 24 16:17 .
drwxr-x--- 3 haru haru 4096  8月 24 16:17 ..
drwxr-x--- 2 haru haru 4096  8月 24 16:17 docs
drwxr-x--- 2 haru haru 4096  8月 24 16:17 tmp

work/25020007046/input/docs:
总计 16
drwxr-x--- 2 haru haru 4096  8月 24 16:17 .
drwxr-x--- 4 haru haru 4096  8月 24 16:17 ..
-rw-r----- 1 haru haru  11  8月 24 16:17 'notes one.txt'
-rw-r----- 1 haru haru   6  8月 24 16:17 .secret.txt

work/25020007046/input/tmp:
总计 8
drwxr-x--- 2 haru haru 4096  8月 24 16:17 .
drwxr-x--- 4 haru haru 4096  8月 24 16:17 ..
-rw-r----- 1 haru haru   0  8月 24 16:17 empty.txt
haru@haru-VMware-Virtual-Platform:~/q01$ find work/25020007046 -type f -printf "%P %s\n" > inventory.txt
haru@haru-VMware-Virtual-Platform:~/q01$ cat inventory.txt
input/tmp/empty.txt 0
input/docs/notes one.txt 11
input/docs/.secret.txt 6
```

图 2 q01 文件复制、权限与清单结果

1.2 第 2 题：随机访问日志统计

主题：Shell 管道与文本处理

1.2.1 任务说明

编写 `analyze.sh`，对 CSV 日志进行统计：输出 HTTP 5xx 数量最多的前 2 个 path，并计算全部数据行的平均延迟；同时处理输入文件不存在的异常情况。

1.2.2 脚本实现

```
1  #!/bin/bash
2
3  if [ ! -f "$1" ]; then
4      echo "Error: file not found: $1" >&2
5      exit 1
6  fi
7
8  echo "Top 2 5xx paths:"
9  awk -F',' 'NR > 1 && $4 >= 500 && $4 < 600 {
10      count[$3]++
11  } END {
```

```

12     for (p in count) print count[p], p
13 }' "$1" | sort -k1,1nr -k2,2 | head -2
14
15 echo "Average latency_ms:"
16 awk -F',' 'NR > 1 {
17     sum += $5; count++
18 } END {
19     printf "%.2f\n", sum/count
20 }' "$1"

```

关键点:

- 使用 if 判断是否是文件，fi 结束判断。
- \$1 表示脚本接收的第一个参数；>&2 把错误信息输出到 stderr，exit 1 表示脚本执行失败，可使用 echo \$? 查看错误输出。
- awk -F',' 用逗号分隔 CSV 字段，NR>1 用来跳过表头。
- CSV 每一行有 5 列：\$1=09:00:05 \$2=bob \$3=/api/orders \$4=502 \$5=260。
- count[\$3]++ 表示计数将路径计数一次。for(p in count): 把 count 里面的每一个 path 都拿出来；print count[p], p: 打印 path 的次数和 path。
- 管道符 | 会把前一个命令的输出交给下一个命令，本题依次使用 awk、sort 和 head 完成统计与排序。sort -k1,1nr -k2,2 表示依据第一列降序排序；head -2 只取前两行。
- 运行 ./analyze.sh access.csv 时，.analyze.sh 是运行脚本，access.csv 是传给脚本的第一个参数；默认参数使用 "\$1"。

1.2.3 实验结果

```

haru@haru-VMware-Virtual-Platform:~/q02$ cat > analyze.sh
#!/bin/bash

if [ ! -f "$1" ]; then
    echo "Error: file not found: $1" >&2
    exit 1
fi

echo "Top 2 5xx paths:"
awk -F',' 'NR > 1 && $4 >= 500 && $4 < 600 {count[$3]++} END {for (p in count) print count[p], p}' "$1" | sort -k1,1nr -k2,2 | head -2

echo "Average latency_ms:"
awk -F',' 'NR > 1 {sum += $5; count++} END {printf "%.2f\n", sum/count}' "$1"
haru@haru-VMware-Virtual-Platform:~/q02$ ./analyze.sh access.csv
Top 2 5xx paths:
3 /api/orders
2 /api/users
Average latency_ms:
193.75

```

图 3 q02 正常日志统计结果

```
haru@haru-VMware-Virtual-Platform:~/q02$ ./analyze.sh not_exist.csv
Error: file not found: not_exist.csv
haru@haru-VMware-Virtual-Platform:~/q02$ echo $?
1
```

图 4 q02 异常输入处理结果

1.3 第 3 题：制造、解决并解释一次合并冲突

主题：Version Control and Git

1.3.1 任务说明

从零初始化 Git 仓库，通过两个分支对同一行进行不同修改，主动制造合并冲突；随后按题目要求解决冲突，并检查最终提交历史。

1.3.2 关键过程

使用 git init 初始化仓库；使用 git status 查看仓库情况，可发现正在 main 分支上；使用 git add 将文件转入暂存区；使用 git commit -m(提交说明) 提交。

```
git init
git status
echo "mode=normal" > config.txt
git add config.txt
git commit -m "initial commit"
```

使用 git switch -c feature-x 创建两条新的分支；使用 git switch 可切换分支；在 feature-a 分支上将文件改为 mode=safe，进行 git add 以及 git commit；在分支 feature-b 上将文件改为 mode=fast；返回 main 分支先合并 feature-a，git merge feature-a；在合并 feature-b 出现合并冲突。

```
git switch -c feature-a
echo "mode=safe" > config.txt
git add config.txt
git commit -m "set mode to safe"
git switch main
git switch -c feature-b
git add config.txt
git commit -m "set mode to fast"
git switch main
git merge feature-a
git merge feature-b
```

冲突后文件内容变为：

```
<<<<<< HEAD
mode=safe
=====
```

```
mode=fast
>>>>>> feature-b
```

发生冲突后，将 config.txt 最终修改为：

```
cat > config.txt
mode=safe
note=reviewed
git add config.txt
git commit -m "merge feature-b and resolve conflict"
```

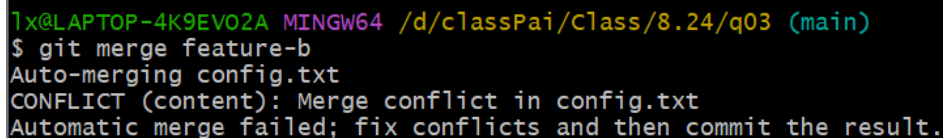
完成冲突解决后检查提交图：

```
git status
git log --all --graph --decorate --oneline
```

关键点：

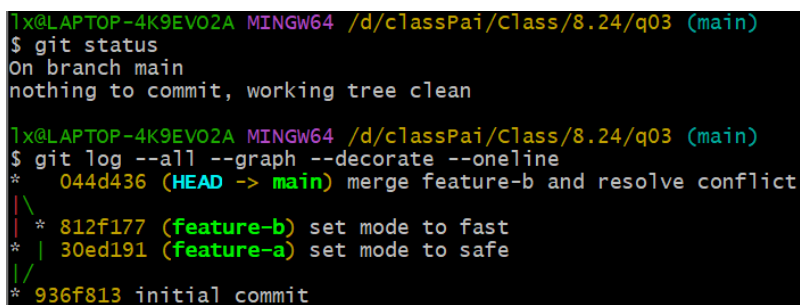
- Git 的基本过程可以理解为“工作区 → 暂存区 (add) → 提交历史 (commit)”。
- 两个分支修改同一位置时可能产生冲突，需要人工决定最终内容，并删除冲突标记。
- git status 用来检查工作区状态，git log --all --graph --decorate --oneline 用来查看分支和提交历史。

1.3.3 实验结果



```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q03 (main)
$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
```

图 5 q03 合并冲突产生过程



```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q03 (main)
$ git status
On branch main
nothing to commit, working tree clean

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q03 (main)
$ git log --all --graph --decorate --oneline
* 044d436 (HEAD -> main) merge feature-b and resolve conflict
|
| * 812f177 (feature-b) set mode to fast
| * 30ed191 (feature-a) set mode to safe
|/
* 936f813 initial commit
```

图 6 q03 冲突解决后的提交历史

1.4 第 4 题：修复并构建一页技术说明

主题：LaTeX 文档编辑

1.4.1 任务说明

在给定 LaTeX 模板中补充带编号与标签的公式、带标题与标签的 2 行 2 列表格，并在正文中完成交叉引用，最后从命令行构建 PDF。

1.4.2 核心实现

公式示例：

$$y = ax + b$$

(1)

正文可使用`\eqref{eq:linear-demo}` 引用公式 (1)。

表 1 示例实验数据

项目	数值
A	10

正文可使用`\ref{tab:demo}` 引用表 1。

1.4.3 构建与结果

```
latexmk -xelatex -halt-on-error report.tex
```

关键点：

- `label` 用来给公式或表格设置标签，`ref` 和 `eqref` 用来在正文中引用编号。
- 表格中的 `caption` 用来生成标题和编号，`label` 一般放在它后面。
- 中文文档使用 `ctexart`，再通过 `latexmk -xelatex` 完成编译和交叉引用更新。

系统开发工具基础实验说明

姬艺华 – 25020007046

1 实验结果

本实验使用一个简单的线性关系作为示例。公式 (1) 给出了变量之间的关系：

$$y = ax + b$$

(1)

其中， a 表示斜率， b 表示截距。

表 1 给出了示例实验数据。

表 1: 实验数据

项目	数值
A	10

图 7 q04 公式、表格与交叉引用结果

```
Output written on report.xdv (1 page, 3364 bytes).
Transcript written on report.log.
Latexmk: Getting log file 'report.log'
Latexmk: Examining 'report.fls'
Latexmk: Examining 'report.log'
Latexmk: Log file says output to 'report.xdv'
Latexmk: applying rule 'xdvipdfmx'...
Rule 'xdvipdfmx': Reasons for rerun
Category 'other':
  Rerun of 'xdvipdfmx' forced or previously required:
    Reason or flag: 'go_mode'

-----
Run number 1 of rule 'xdvipdfmx'
-----
Running 'xdvipdfmx -E -o "report.pdf" "report.xdv"'
-----
report.xdv -> report.pdf
[1]
25592 bytes written
Latexmk: All targets (report.xdv report.pdf) are up-to-date
```

图 8 q04 命令行构建结果

2 课下习题

本部分整理课后完成的 11 个 Shell 练习。为保持版面简洁，每个练习只保留题目名称、关键命令、简要说明和截图位置。

2.1 检查 Shell 环境

```
echo $SHELL
```

输出显示当前使用 Bash，符合实验要求。

```
lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test
$ echo $SHELL
/usr/bin/bash
```

图 9 检查 Shell 环境

2.2 ls -l 与文件权限

```
ls -l /
```

每行前 10 个字符表示文件类型和权限。第 1 位表示文件类型，后面 9 位依次表示所有者、用户组和其他用户的读、写、执行权限。

```
lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test
$ ls -l /
total 8352
-rw-r--r-- 1 lx 197121 18765 Feb 2 2026 LICENSE.txt
-rw-r--r-- 1 lx 197121 300037 Feb 2 2026 ReleaseNotes.html
drwxr-xr-x 1 lx 197121 0 Feb 23 2026 bin/
drwxr-xr-x 1 lx 197121 0 Feb 23 2026 cmd/
drwxr-xr-x 1 lx 197121 0 Feb 23 2026 dev/
drwxr-xr-x 1 lx 197121 0 Feb 23 2026 etc/
-rwxr-xr-x 1 lx 197121 138656 Feb 2 2026 git-bash.exe*
-rwxr-xr-x 1 lx 197121 138128 Feb 2 2026 git-cmd.exe*
drwxr-xr-x 1 lx 197121 0 Feb 23 2026 mingw64/
dr-xr-xr-x 9 lx 197121 0 Aug 27 22:39 proc/
drwxr-xr-x 1 lx 197121 0 Aug 27 22:37 tmp/
-rw-r--r-- 1 lx 197121 1994356 Feb 23 2026 unins000.dat
-rwxr-xr-x 1 lx 197121 4441856 Feb 23 2026 unins000.exe*
-rw-r--r-- 1 lx 197121 25388 Feb 23 2026 unins000.msg
drwxr-xr-x 1 lx 197121 0 Feb 23 2026 usr/
```

图 10 使用 ls -l 查看文件权限

2.3 Glob 匹配

```
ls *.txt
ls file?.txt
ls {a,b,c}.txt
```

* 可以匹配任意数量字符，? 匹配一个字符，{a,b,c} 可以展开为多个文件名。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test
$ mkdir glob_test

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test
$ cd glob_test

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ touch file1.txt file2.txt file10.txt a.txt b.txt c.txt hello.md

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ ls
a.txt b.txt c.txt file1.txt file10.txt file2.txt hello.md

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ ls *.txt
a.txt b.txt c.txt file1.txt file10.txt file2.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ ls file?.txt
file1.txt file2.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ ls {a,b,c}.txt
a.txt b.txt c.txt
```

图 11 Glob 文件名匹配结果

2.4 Shell 引号

```
echo '$name'
echo "$name"
printf '%s\n' '$Price: $100!\nHello'
```

单引号中的内容基本按原样输出；双引号允许变量展开；`$'...'` 形式可以识别 `\n` 等转义字符。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ printf '%s\n' '$Price: $100!\nHello'
Price: $100!
Hello

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ name=Tom
echo '$name'
echo "$name"
$name
Tom
```

图 12 Shell 不同引号的输出结果

2.5 标准流重定向

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
ls /nonexistent /tmp > all.txt 2>&1
```

> 重定向标准输出，2> 重定向标准错误，2>&1 可以把两者写入同一个文件。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ ls /nonexistent /tmp
ls: cannot access '/nonexistent': No such file or directory
/tmp:
000e629c-2c0c-44c0-a679-9c648e20b9fb/
017c37a8-480b-451e-8aa0-b16fe5da030a.tmp.png
01df5a6c-b621-4376-a0c3-2a86d89bef8a.tmp.js
043dc2eb-3e8f-45d7-b783-64461bb93100.tmp.js
04a45f35-5c43-447e-b4c1-8b9fff0d4cb5.tmp.js
052ac38a-d2ea-4f61-b094-92a91dcc2f9b.tmp.js
053a8ce1-f726-4561-814b-5ac48c1ee875.tmp.js
056ce6bb-8c03-45bf-a0f8-075f5531b8d2.tmp.js
06140164-1cd3-41fe-a1ff-38673446ae9a.tmp.js
0718a2eb-e57b-4b93-bc44-90f903702fe7/
```

图 13 标准输出与标准错误分开重定向

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ ls /nonexistent /tmp > stdout.txt 2> stderr.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ cat stdout.txt
/tmp:
000e629c-2c0c-44c0-a679-9c648e20b9fb/
017c37a8-480b-451e-8aa0-b16fe5da030a.tmp.png
01df5a6c-b621-4376-a0c3-2a86d89bef8a.tmp.js
043dc2eb-3e8f-45d7-b783-64461bb93100.tmp.js
04a45f35-5c43-447e-b4c1-8b9fff0d4cb5.tmp.js
052ac38a-d2ea-4f61-b094-92a91dcc2f9b.tmp.js
053a8ce1-f726-4561-814b-5ac48c1ee875.tmp.js
056ce6bb-8c03-45bf-a0f8-075f5531b8d2.tmp.js
06140164-1cd3-41fe-a1ff-38673446ae9a.tmp.js
0718a2eb-e57b-4b93-bc44-90f903702fe7/
```

图 14 查看标准输出和标准错误文件

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ cat stderr.txt
ls: cannot access '/nonexistent': No such file or directory

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ ls /nonexistent /tmp > all.txt 2>&1

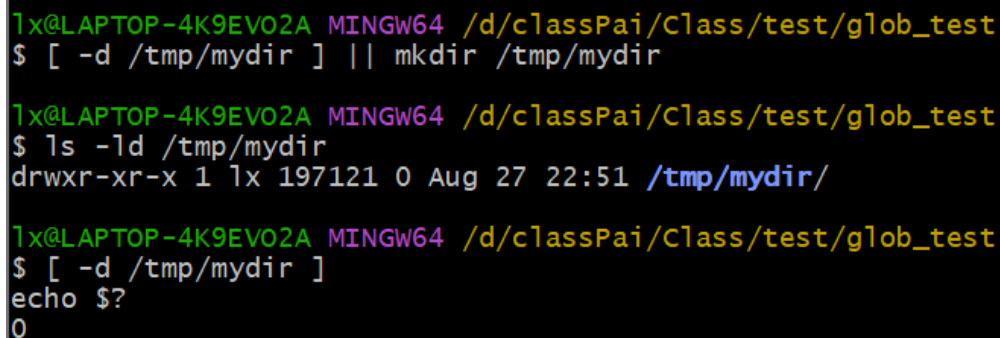
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ cat all.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
000e629c-2c0c-44c0-a679-9c648e20b9fb/
017c37a8-480b-451e-8aa0-b16fe5da030a.tmp.png
01df5a6c-b621-4376-a0c3-2a86d89bef8a.tmp.js
043dc2eb-3e8f-45d7-b783-64461bb93100.tmp.js
04a45f35-5c43-447e-b4c1-8b9fff0d4cb5.tmp.js
052ac38a-d2ea-4f61-b094-92a91dcc2f9b.tmp.js
053a8ce1-f726-4561-814b-5ac48c1ee875.tmp.js
056ce6bb-8c03-45bf-a0f8-075f5531b8d2.tmp.js
06140164-1cd3-41fe-a1ff-38673446ae9a.tmp.js
0718a2eb-e57b-4b93-bc44-90f903702fe7/
```

图 15 标准输出与标准错误合并重定向

2.6 || 与退出状态

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir  
echo $?
```

只有当前面的判断失败，也就是目录不存在时，才会执行 `mkdir`。`$?` 可以查看上一条命令的退出状态。



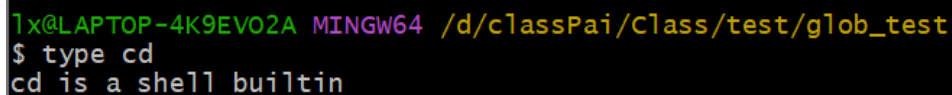
```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test  
$ [ -d /tmp/mydir ] || mkdir /tmp/mydir  
  
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test  
$ ls -ld /tmp/mydir  
drwxr-xr-x 1 1x 197121 0 Aug 27 22:51 /tmp/mydir/  
  
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test  
$ [ -d /tmp/mydir ]  
echo $?  
0
```

图 16 || 与退出状态实验结果

2.7 cd 为什么是内建命令

```
type cd
```

结果显示 `cd` 是 Shell 内建命令。因为子进程不能改变父进程 Shell 的当前目录，所以 `cd` 必须由 Shell 自己执行。



```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test  
$ type cd  
cd is a shell builtin
```

图 17 查看 `cd` 的命令类型

2.8 判断文件是否存在

```
1 #!/bin/bash  
2 if [ -f "$1" ]; then  
3     echo "File exists."  
4 else  
5     echo "File does not exist."  
6 fi
```

脚本通过 `$1` 获取文件名，并使用 `-f` 判断文件是否存在。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ cat > check.sh <<'EOF'
#!/bin/bash

if [ -f "$1" ]; then
    echo "File exists."
else
    echo "File does not exist."
fi
EOF

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ cat check.sh
#!/bin/bash

if [ -f "$1" ]; then
    echo "File exists."
else
    echo "File does not exist."
fi

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ bash check.sh a.txt
File exists.

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test
$ bash check.sh abc.txt
File does not exist.
```

图 18 脚本判断文件是否存在

2.9 文件执行权限

```
chmod +x check.sh
./check.sh somefile
```

没有执行权限时直接运行脚本会失败；添加 `x` 权限后，可以使用 `./check.sh` 直接执行。

```
haru@haru-VMware-Virtual-Platform:~/q02$ cd ~
haru@haru-VMware-Virtual-Platform:~$ cat > check.sh <<'EOF'
#!/bin/bash

if [ -f "$1" ]; then
    echo "File exists."
else
    echo "File does not exist."
fi
EOF
haru@haru-VMware-Virtual-Platform:~$ ls -l check.sh
-rw-rw-r-- 1 haru haru 98  8月 27 23:01 check.sh
haru@haru-VMware-Virtual-Platform:~$ ./check.sh somefile
bash: ./check.sh: 权限不够
haru@haru-VMware-Virtual-Platform:~$ chmod +x check.sh
haru@haru-VMware-Virtual-Platform:~$ ls -l check.sh
-rwxrwxr-x 1 haru haru 98  8月 27 23:01 check.sh
haru@haru-VMware-Virtual-Platform:~$ ./check.sh somefile
File does not exist.
```

图 19 脚本执行权限实验结果

2.10 set -x

```
set -x
```

加入 `set -x` 后，运行脚本时会显示实际执行的命令，可用于调试 Shell 脚本。

```
haru@haru-VMware-Virtual-Platform:~/q02$ cd ~
haru@haru-VMware-Virtual-Platform:~$ cat > check.sh <<'EOF'
#!/bin/bash

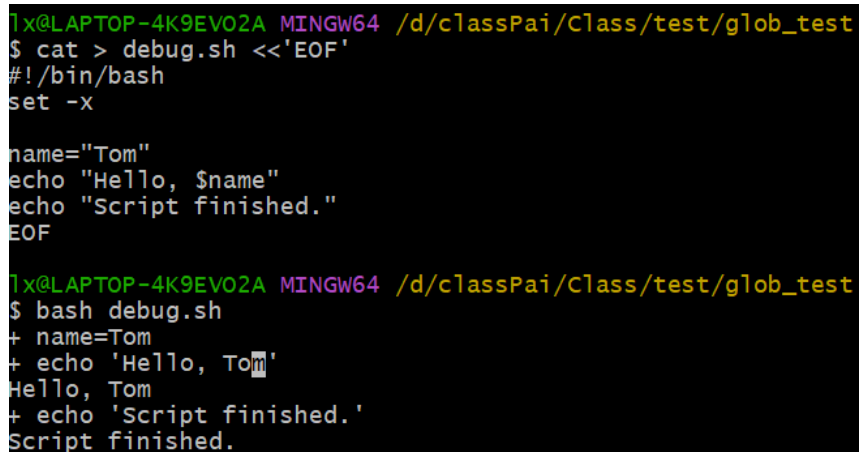
if [ -f "$1" ]; then
    echo "File exists."
else
    echo "File does not exist."
fi
EOF
haru@haru-VMware-Virtual-Platform:~$ ls -l check.sh
-rw-rw-r-- 1 haru haru 98  8月 27 23:01 check.sh
haru@haru-VMware-Virtual-Platform:~$ ./check.sh somefile
bash: ./check.sh: 权限不够
haru@haru-VMware-Virtual-Platform:~$ chmod +x check.sh
haru@haru-VMware-Virtual-Platform:~$ ls -l check.sh
-rwxrwxr-x 1 haru haru 98  8月 27 23:01 check.sh
haru@haru-VMware-Virtual-Platform:~$ ./check.sh somefile
File does not exist.
```

图 20 使用 `set -x` 调试脚本

2.11 -- 的作用

```
touch -- -myfile  
rm ./-myfile
```

-- 表示后面的内容不再作为命令选项处理。对于以 - 开头的文件名，也可以使用 ./ 文件名的方式操作。



```
ix@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test  
$ cat > debug.sh <<'EOF'  
#!/bin/bash  
set -x  
  
name="Tom"  
echo "Hello, $name"  
echo "Script finished."  
EOF  
  
ix@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/test/glob_test  
$ bash debug.sh  
+ name=Tom  
+ echo 'Hello, Tom'  
Hello, Tom  
+ echo 'Script finished.'  
Script finished.
```

图 21 使用 -- 处理特殊文件名

3 Git 与作业提交

3.1 仓库内容整理

```
8.24/  
|-- q01/  
|-- q02/  
|-- q03/  
|-- q04/  
`-- lab_report/  
    |-- figures/  
    |-- lab_report.tex  
    `-- lab_report.pdf
```

3.2 GitHub / Gitee 提交记录

仓库地址: <https://github.com/haru-senya/8.24.git>

3.3 commit 记录截图、仓库页面截图

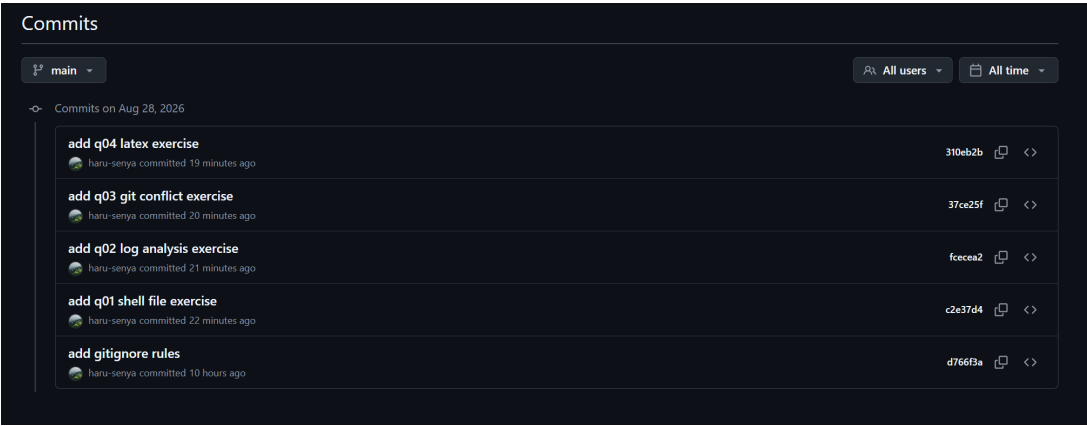


图 22 Git commit 记录

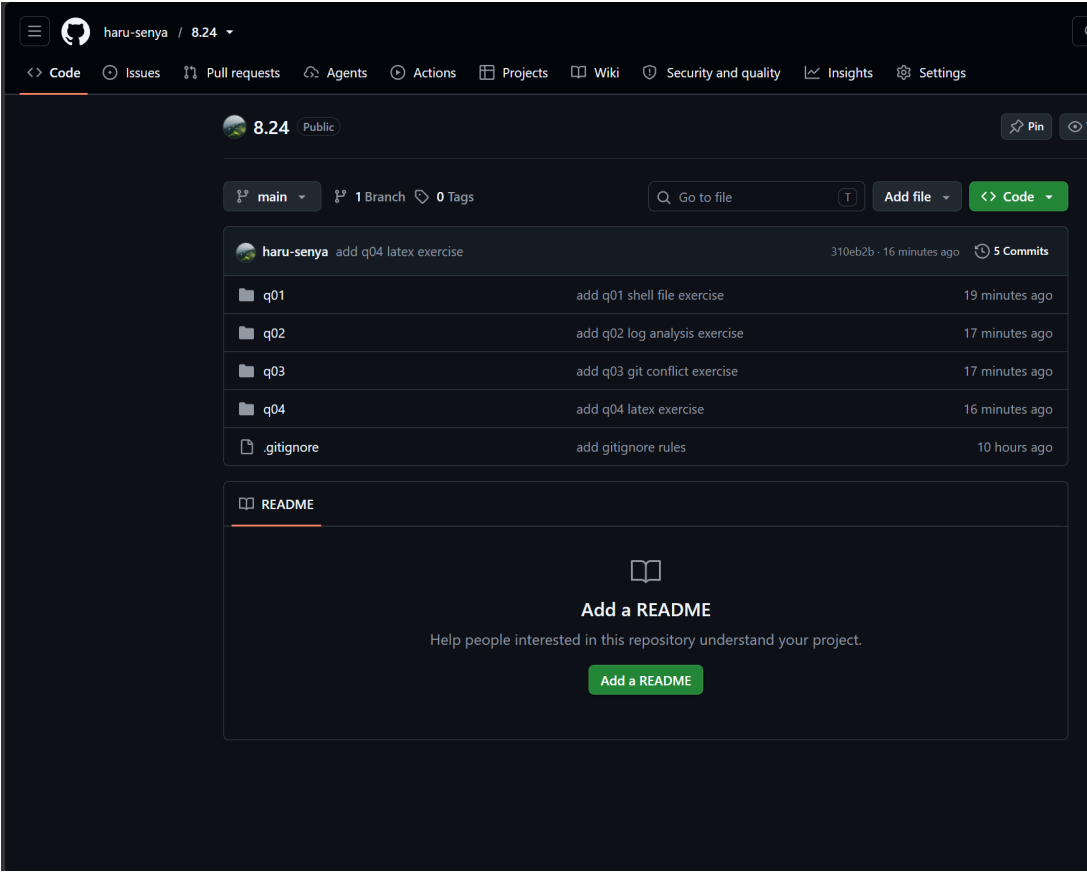


图 23 GitHub / Gitee 仓库页面

4 问题以及经验

4.1 实验中遇到的问题

虚拟机文件迁移：q01 与 q02 最初保存在 Ubuntu 虚拟机中，VMware 直接拖拽失败后，改用共享文件夹将实验目录复制到 Windows 主机中。

LaTeX 中文编译：使用 pdfLaTeX 编译中文时出现问题，因此改用 ctexart 文档类和 Xe-

LaTeX；配置好 Perl 后，LaTeX Workshop 可以通过 `latexmk` 自动构建和预览。

4.2 实验经验

本次实验让我更熟悉了 Shell、Git 和 LaTeX 的基本使用。Shell 部分需要注意参数、引号、权限和重定向等细节；Git 可通过“工作区-暂存区-提交历史”来理解；LaTeX 则适合用统一规则管理文档结构和排版。

4.3 总结

通过课上题和课下练习，我完成了文件操作、文本处理、Git 冲突解决和 LaTeX 文档构建等任务，也对开发工具的基本工作方式有了初步认识。后续还需要在实际使用中继续熟悉这些工具。